

CSG Sweeps: Feature-aware sweep surfacing of deformable CSGs

JIANJUN XIA, Washington University in St. Louis, USA

QINGNAN ZHOU, Adobe Research, USA

YIWEN JU, Washington University in St. Louis; Stability AI, USA

XINGYI DU, Tencent, USA

NATHAN CARR, Adobe Research, USA

TAO JU, Washington University in St. Louis, USA

Computing the sweep surface of a moving 3D solid is a fundamental problem in computer graphics. The problem is particularly challenging when the solid is piecewise-smooth, which yields a complex network of sharp features on the sweep surface. Few methods can model these features, and they are all limited to rigid-body motions (translation and rotation). We present the first feature-aware method to surface the sweep of a deforming piecewise-smooth shape. Our method works on any input represented as a 4D time-varying CSG, allowing changes to both geometry and topology during sweeping. Our technical contributions include a generalization of the lifted sweeping formulation from smooth functions to CSG functions, and robust algorithms for discretizing multiple intersecting implicit surfaces in arbitrary dimensions. We demonstrate our algorithm for capturing the sharp features on the sweep surface under both rigid and non-rigid motions.

CCS Concepts: • **Computing methodologies** → **Volumetric models; Mesh models.**

Additional Key Words and Phrases: Sweep volume, implicit surfaces, CSG, arrangement

ACM Reference Format:

Jianjun Xia, Qingnan Zhou, Yiwen Ju, Xingyi Du, Nathan Carr, and Tao Ju. 2026. CSG Sweeps: Feature-aware sweep surfacing of deformable CSGs. *ACM Trans. Graph.* 45, 6, Article 185 (December 2026), 14 pages. <https://doi.org/10.1145/3842580>

1 Introduction

The *sweep surface* of a moving shape is the boundary of the volume swept by the shape. Such surfaces are useful in a wide range of application domains. In engineering, sweep surfaces help design moving parts (e.g., gears) that fit each other in an assembly. In manufacturing, the simulation of machining relies on accurate modeling of the sweep surface of the milling tool. In robotics, sweep surfaces are useful for designing collision-free paths. Sweeping is also a common modeling metaphor used by designers and artists.

In most of these applications, sweeping is performed on *man-made* shapes, which are characterized by a piecewise-smooth boundary containing sharp edges and corners. The resulting sweep surface is also piecewise smooth and often rich in sharp features, many of which are swept by the sharp features on the shape (see Figure 1).

Authors' Contact Information: Jianjun Xia, Washington University in St. Louis, USA, jianjun.x@wustl.edu; Qingnan Zhou, Adobe Research, USA, qzhou@adobe.com; Yiwen Ju, Washington University in St. Louis; Stability AI, USA, yiwen.ju@wustl.edu; Xingyi Du, Tencent, USA, xingyidu@global.tencent.com; Nathan Carr, Adobe Research, USA, ncarr@adobe.com; Tao Ju, Washington University in St. Louis, USA, taoju@wustl.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 1557-7368/2026/12-ART185
<https://doi.org/10.1145/3842580>

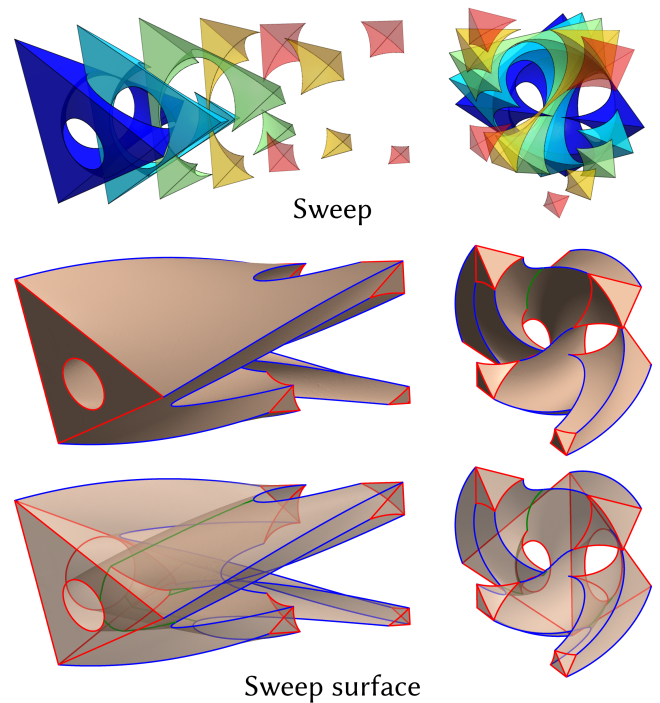


Fig. 1. A deforming CSG shape (top, a moving tetrahedron subtracted by a growing sphere), showing frames in two views colored by time, and the sweep surface computed by our method (bottom). Red, blue, and green curves are shape edges at the start and finish of the sweep, creases swept by the shape corners, and self-intersections of the sweep volume.

These features not only enhance the visual fidelity of the sweep surface, but are also important for feature analysis and downstream feature-aware design.

While sweeping has been extensively studied in the past, only a few methods are designed for piecewise-smooth shapes, and even fewer can model the sharp edges on the sweep surface [Adsul et al. 2016; Conkey and Joy 2000; Rossignac et al. 2007]. These methods are all limited to sweeping shapes under rigid-body motions (translation and rotation). However, shapes often deform while sweeping in practical scenarios. For example, tools may warp under thermal and mechanical stress during manufacturing, robots may bend at joints as they move, and reconfigurable structures may extend or retract. Allowing deformation in sweep-based modelling can also offer designers and artists a larger design space. Although methods

exist for sweeping deformable shapes [Blackmore et al. 1994; Erdim and Ilieş 2008; Ju et al. 2025], none can capture the sharp features resulting from sweeping a piecewise-smooth shape.

In this paper, we propose an algorithm for sweeping a piecewise-smooth 3D shape that not only models the sharp features on the sweep surface but also accommodates non-rigid shape deformations. We consider any sweep defined by a Constructive Solid Geometry (CSG) in the 4-dimensional, spatial-temporal domain, which applies set operators (union, intersection, difference) on a group of moving and deforming smooth shapes. The flexible representation allows changes in both the geometry and topology of the piecewise-smooth shape over time. Our method produces a watertight mesh equipped with the various types of sharp features of the sweep surface, including edges and corners of the piecewise-smooth shape at the start and end of sweeping, creases swept by the sharp corners of the shape, and the self-intersections of the sweep volume (see Figure 1).

Our method builds upon a recent method for sweeping a deforming *smooth* shape by Ju et al. [2025]. Using a *lifted* formulation, this method can model the sharp creases where the sweep volume intersects itself by first computing a smooth surface in 4D (called the *lifted envelope*) and trimming its 3D projection (the *envelope*). We extend their method to sweep a *piecewise-smooth* deforming shape and further capture the sharp edges on the sweep surface originating from the sharp features on the shape. The extension involves the following contributions in theory and algorithms:

- (1) Theoretically, we extend the lifted formulation in [Ju et al. 2025] from a smooth function to a piecewise-smooth function (Section 4). Specifically, we introduce a new component on the lifted envelope, called the “sharp contours” (Section 4.2), generalize the winding-number-based characterization of the sweep surface (Section 4.3), and analyze the sharp features on the sweep surface (Section 4.4).
- (2) Algorithmically, to compute the piecewise-smooth lifted envelope in 4D, we design a general algorithm for computing the arrangement of implicit surfaces in arbitrary dimensions (Section 6.1), together with a robust treatment for zero-level sets of pairwise difference functions (Section 6.2). We also extend the 4D grid refinement algorithm of [Ju et al. 2025] to a piecewise-smooth function (Section 7).

2 Related works

2.1 Sweeping

We briefly review existing methods for computing sweep surfaces, with an eye towards sharp features and deformable shapes.

2.1.1 Rigid sweeping of smooth shapes. The structure of the sweep surface of a smooth shape in rigid motion is well-understood [Blackmore and Leu 1992; Martin and Stephenson 1990]. The sweep surface is a subset of the *envelope*, a self-intersecting surface made up of *characteristic curves*, which in turn consists of surface points whose velocity vectors are tangent to the surface. The actual computation, however, is far from straightforward, as it involves solving systems of nonlinear equations. Precise, analytical surfaces are only known for simple classes of surfaces [Abdel-Malek and Yeh 1997; Rabl et al.

2008; Wang and Wang 1986]. Methods for handling more general shapes typically compute characteristic curves at discrete times and connect them into surfaces by point-based surfacing [Blackmore and Leu 1992; Blackmore et al. 1997; Peternell et al. 2005], forming triangle strips [Yang and Abdel-Malek 2005], or approximating by parametric surfaces [Adsul et al. 2015; Weinert et al. 2004; Xu et al. 2007]. While these methods can identify sharp intersections of the envelope, connecting characteristic curves and trimming the envelope are highly non-trivial for complex shapes and motions.

Alternatively, the sweep surface can be formulated implicitly as the level set of a distance field. A straightforward way to obtain this field is by combining the distance fields at discrete time stamps [Schroeder et al. 1994; Sourin and Pasko 1995], but this *stamping* approach creates notable artifacts on the level set. Smoother and more accurate distances can be computed by intersecting the reverse trajectories of grid points with the shape [Erdim and Ilieş 2008, 2010; Ilies and Shapiro 1999; Wang et al. 2025] or continuously tracking the sweep surface in space-time [Sellán et al. 2021]. While the implicit methods guarantee a water-tight surface, the robustness comes at the cost of losing sharp features due to the discretization of the implicit surface (see results of [Sellán et al. 2021] in Figure 2 (b)).

2.1.2 Sweeping piecewise-smooth shapes. The sweep surface of a piecewise-smooth shape has a richer set of sharp features than sweeping smooth shapes. Besides self-intersections of the envelope, corners of the shape may sweep to form sharp edges on the sweep surface, and some sharp features of the shape at the start and end of the sweep may remain on the sweep surface (see Figure 2 (d)). While a few methods can model all these features, most of them are designed for limited classes of shapes, such as polyhedra made up of flat polygons [Abrams and Allen 1995; Campen and Kobbelt 2010; Weld and Leu 1990], trivariate Bezier solids [Conkey and Joy 2000; Madrigal and Joy 1999], and simple B-rep shapes [Abdel-Malek et al. 2001; Kieffer and Litvin 1991]. Only a handful of methods can deal with general B-reps, either in screw motion [Rossignac et al. 2007] or arbitrary rigid motion [Adsul et al. 2016; Machchhar et al. 2017].

All of these methods work by computing, trimming, and connecting the trajectory of individual boundary elements. This approach has inherent difficulties in handling a deforming sweep, where both the geometry and connectivity of the boundary elements may change with time. Furthermore, no sweeping methods have been designed for CSGs, which are more flexible than B-reps in describing shapes with changing geometry and topology.

2.1.3 Deformable sweeping. Works on sweeping deformable shapes are scarce and often limited to specific classes of deformations, such as affine motions [Erdim and Ilieş 2008, 2010] and diffeomorphisms [Blackmore et al. 1994]. Implicit methods, such as stamping [Schroeder et al. 1994; Sourin and Pasko 1995] and space-time tracking [Sellán et al. 2021], can be potentially applied to arbitrary deformations (invertible or not) defined by a space-time function, but they cannot capture sharp features on the sweep surface.

To our knowledge, [Ju et al. 2025] is the only method that can sweep smooth shapes under arbitrary deformations and produce the sharp creases where the envelope self-intersects (see a review of this method in Section 3). However, it cannot recover the sharp edges

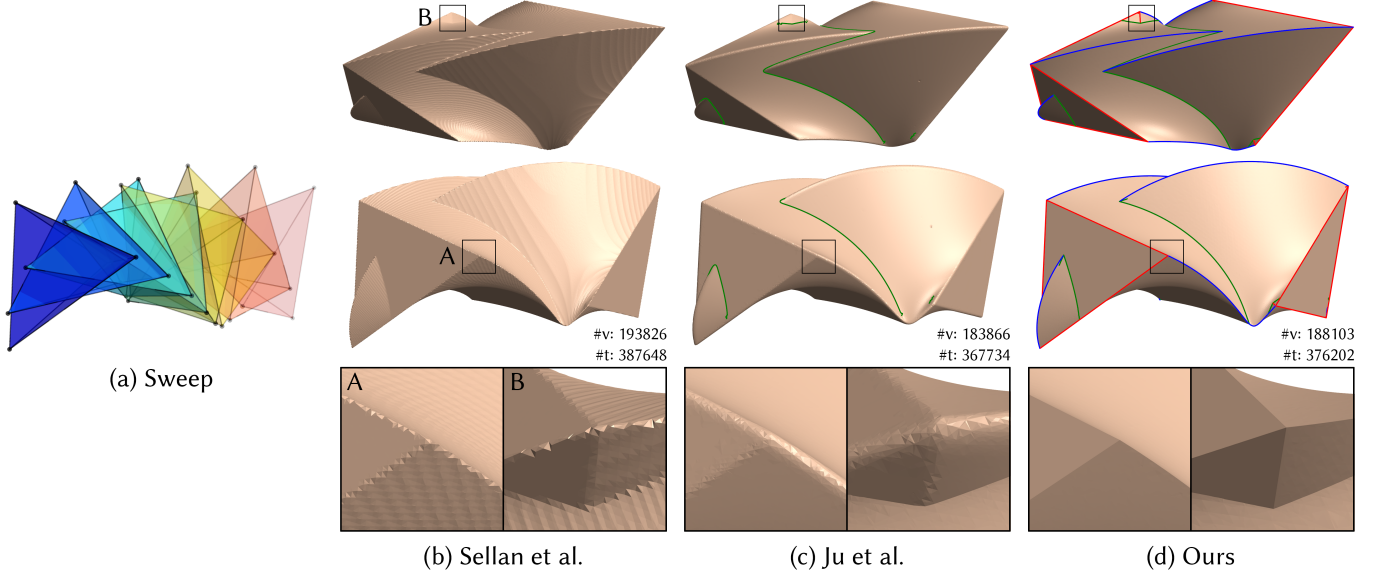


Fig. 2. Comparing [Sellán et al. 2021] (b), [Ju et al. 2025] (c), and ours (d) on sweeping a rolling tetrahedron (a). The surface of [Sellán et al. 2021] has no sharp features, and [Ju et al. 2025] only retains the sharp edges where the envelope self-intersects (green curves). Our method additionally captures sharp edges swept by the corners of the tetrahedron (blue), as well as those on the shape at the start and finish of the sweep (red).

on the sweep surface swept by the sharp features of a piecewise-smooth shape (see Figure 2 (c)). We build on this method to capture the full set of sharp features on the sweep surface (Figure 2 (d)) while retaining the ability to handle arbitrary deformations.

2.2 Computing implicit surfaces and arrangement

Extracting level sets of implicit functions, also known as contouring, has been extensively studied [De Araújo et al. 2015]. Most methods are designed for contouring a single function, such as Marching Cubes and Marching Tetrahedra. Some methods extract the intersection of k level sets in $\mathbb{R}^n$, which are generally $(n - k)$ -manifolds [Allgower and Schmidt 1985; Boissonnat et al. 2023], but they do not produce the level set surfaces themselves. Only a handful of methods compute the *arrangement* of multiple implicit surfaces, which is a spatial decomposition that includes the level sets and their intersections [Bagley et al. 2016; Du et al. 2022; Kim et al. 2000]. However, these methods are limited to 3D, as they either rely on tessellation stencils or contouring heuristics tailored for 3D polyhedra. To the best of our knowledge, no existing algorithm can compute the arrangement of multiple implicit surfaces in higher (≥ 4) dimensions. We will present such an algorithm as part of discretizing the silhouette set (Section 6).

3 Lifted sweeping: a review

Our method directly builds on the work by Ju et al. [2025], who define and compute the surface of a smooth sweep by lifting it to a higher dimension. For completeness, we will first briefly review the key results of their method.

We consider a sweep in $\mathbb{R}^n$ defined implicitly by a generic, C^2 -continuous *sweep function* $f(x, t)$, where $x \in \mathbb{R}^n$ and $t \in [0, 1]$, so that $f(x, t) < 0$ if x is inside the shape at time t . The *sweep volume*

is the set of all points x such that $f(x, t) < 0$ for some t . We assume that the sweep volume is bounded, and the boundary is called the *sweep boundary* or *sweep surface*. As a 2D example, Figure 3(a) shows the sweep volume and boundary of a disk moving in a loopy path. We will use this example to explain the concepts below.

3.1 Lifting the sweep

We define the *lifted sweep volume* as the set of all $\{x, t\} \in \mathbb{R}^{n+1}$ where $f(x, t) < 0$. Its boundary, a smooth n -manifold defined by the zero-level set $f(x, t) = 0$, is called the *sweep set* (Figure 3 (c)). The sweep volume in $\mathbb{R}^n$ is the projection of the lifted sweep volume, and the sweep boundary in $\mathbb{R}^n$ is the projection of the “silhouette” of the sweep set. A point $\{x, t\}$ on the silhouette can be classified by local differential properties as one of three types,

- *Contour*: $t \in (0, 1)$, $f(x, t) = 0$, $f'(x, t) = 0$.
- *Bottom cap*: $t = 0$, $f(x, t) = 0$, $f'(x, t) > 0$.
- *Top cap*: $t = 1$, $f(x, t) = 0$, $f'(x, t) < 0$.

Here, $f' = \partial f / \partial t$ is the time derivative. The union of these three types is called the *lifted envelope* (blue and red curves in Figure 3 (c)). Its projection in $\mathbb{R}^n$ is the *envelope*, a superset of the sweep boundary (Figure 3 (b)).

The lifted envelope can also be defined as the intersection of two implicit surfaces: the sweep set, which is the zero-level set of the sweep function f ; and the *silhouette set*, which is the zero-level set of the *silhouette function* g defined as

$$g(x, t) = \begin{cases} f'(x, t), & 0 < t < 1 \\ -1, & t = 0 \\ 1, & t = 1 \end{cases} \quad (1)$$

Like the lifted envelope, the silhouette set can be decomposed into three components, the *contour set* ($f'(x, t) = 0$, $0 < t < 1$), the

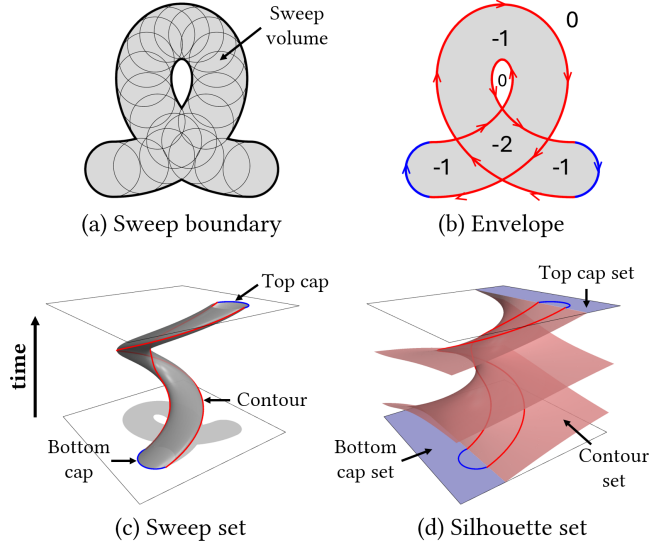


Fig. 3. A smooth sweep. (a) The sweep volume and sweep boundary of a moving circle. (b) The oriented envelope (red/blue curves projected from contour/caps) and the winding number of each arrangement cell. (c) The 3D sweep set (gray surface) and the lifted envelope (red and blue curves). (d) The 3D silhouette set (red and blue surfaces).

bottom cap set ($f'(x, t) > 0, t = 0$), and the top cap set ($f'(x, t) < 0, t = 1$), respectively containing the contour, bottom cap, and top cap of the lifted envelope (Figure 3 (d)).

3.2 Characterizing sweep boundary

Given an envelope, the sweep boundary consists of all envelope points that are not inside the shape at any time during the sweep. Checking if an envelope point lies on the sweep boundary is typically done by intersecting the reverse trajectory of the sweep with the shape [Erdim and Ilieş 2008; Wang et al. 2025]. However, this process is not only time-consuming but also difficult to generalize to deformable sweeps, whose inverse trajectories are either difficult to compute or may not exist.

A key result of [Ju et al. 2025] is that the sweep boundary can be identified efficiently and robustly using the lifted envelope. The lifted envelope is assigned an *outward* orientation. This orientation, projected onto the envelope, defines a *winding number* within each region of the *arrangement* of the envelope. Intuitively, the winding number counts the number of times the oriented envelope “wraps” around a point. The regions with winding number 0 make up the space outside the sweep volume, and their boundary is the sweep boundary (see Figure 3 (b)).

3.3 Computing the sweep

The sweep boundary in 3D is computed in two stages. The first stage computes the lifted envelope in 4D, which is the intersection of two implicit surfaces (sweep set and silhouette set). This stage starts by generating an adaptive 4D grid that is more refined around the lifted envelope. The implicit intersection is computed by first contouring the silhouette function g , obtaining a polyhedral silhouette set, and

then contouring the sweep function f over the polyhedra, resulting in a triangulated lifted envelope.

In the second stage, the lifted envelope is projected to 3D and trimmed to the sweep boundary. Trimming is performed by computing the arrangement of the self-intersecting triangular mesh using an existing package [Zhou et al. 2016]. This package also produces the winding numbers, and the triangles bounding arrangement cells with zero winding form the sweep surface.

4 Theory

We start with the theoretical foundation of our algorithms. We will first introduce our sweep representation (Section 4.1), and then generalize the key results of [Ju et al. 2025] from a smooth sweep to a CSG sweep (Sections 4.2 and 4.3).

4.1 CSG sweep

We define a sweep in $\mathbb{R}^n$ as an implicit CSG in $\mathbb{R}^{n+1}$. Specifically, the sweep function $f(x, t)$ for a point $x \in \mathbb{R}^n$ and time $t \in [0, 1]$ is the result of performing *min* and *max* operations on C^2 -continuous *member functions* $f_i(x, t)$ for $i = 1, \dots, m$. The *sweep volume* is the set of all points x such that $f(x, t) < 0$ for some t . We assume that the sweep volume is bounded, whose boundary is called the *sweep boundary* or *sweep surface*.

Intuitively, the sweep is constructed by applying set operations (e.g., union, intersection, difference) to a group of shapes that move and deform in time. The representation allows the shape to change not only in geometry but also in topology (e.g., connected components and genus). Consider the 2D example in Figure 4 (a), defined as the difference between two moving disks of different sizes. The shape changes its geometry continuously, mimicking an “eclipse” event, and it disappears completely during the “totality”.

It is important to note that the *sweep of a CSG* is not the same as the *CSG of the sweeps* defined by individual member functions. In the example of Figure 4, sweeping the CSG creates a two-part sweep volume, shown in (b). However, subtracting the sweep volume of the larger moving disk from that of the smaller one creates an empty set, as shown in (c).

Similar to [Ju et al. 2025], we make several genericity assumptions about the input. Specifically, (i) the zero-level set of any of the following functions is either empty or an n -manifold: f_i and its time derivative function $f'_i = \partial f_i / \partial t$ for any i , and the difference $f_i - f_j$ for any $\{i, j\}$; (ii) the intersection between any $k \leq n + 1$ implicit surfaces, each being either a zero-level set in (i) or the hyperplane at $t = 0$ or $t = 1$, is either empty or an $(n + 1 - k)$ -manifold; and (iii) the intersection of two implicit surfaces in (ii) have generic projections to $\mathbb{R}^n$, meaning that subset of the intersection where the projection map is not invertible has a dimension of at most $n - 2$. These assumptions exclude sweeps where some member function f_i is stationary, which violates (i) because f'_i is zero everywhere, or where two member functions differ by a constant multiplier, which violates (ii) because their zero-level sets are identical.

4.2 Lifting the sweep

We follow [Ju et al. 2025] and consider the CSG sweep volume in $\mathbb{R}^n$ as the projection of a *lifted sweep volume* in $\mathbb{R}^{n+1}$ consisting of all

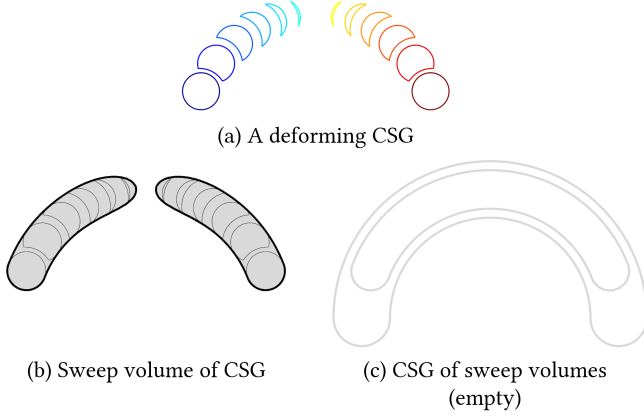


Fig. 4. A deforming CSG defined by subtracting a bigger moving disk from a smaller one (a) produces a sweep volume (b), while the difference between the sweep volumes of the two disks is empty (c).

space-time points $\{x, t\}$ where $f(x, t) < 0$. This volume is bounded by the *sweep set*, an n -manifold in $\mathbb{R}^{n+1}$ (due to genericity) defined by the zero-level set $f(x, t) = 0$.

Unlike in a smooth sweep considered by [Ju et al. 2025], the sweep set in a CSG sweep is only *piecewise-smooth*. It is made up of smooth pieces, each lying on the level set of a single member function, that meet sharply (with C^0 continuity). Consider the 2D example in Figure 5 (a), defined by intersecting two moving disks. The sweep volume of this rolling “lens” is shown in (b), and its lifting in (d). Observe that the cylindrical sweep set in (d) consists of two smooth patches, each being the trajectory of an arc of the lens as it travels over time. The two patches meet along two sharp creases, which are the trajectories of the two sharp corners of the lens.

Lifted envelope. [Ju et al. 2025] formulates the sweep boundary by projecting and trimming the silhouette of the sweep set, known as the *lifted envelope*. As the sweep set of a CSG sweep is only piecewise-smooth, some silhouette points may lie on the sharp creases of the sweep set (e.g., green curves in Figure 5 (d)). Note that the time derivative of the sweep function f' is undefined on those creases. To identify such silhouette points, we observe that the two member functions whose level sets meet there must have *opposite* signs in their time derivatives.

Specifically, we say f_i is *dominant* at a space-time point $\{x, t\}$ if $f_i(x, t) = f(x, t)$. Each smooth patch of the sweep set lies on the level set of a single dominant function, which is an n -manifold due to genericity. The crease where k smooth patches meet has k dominant functions, and it forms an $(n + 1 - k)$ -manifold. Denote by $\alpha(x, t)$ the indices of the dominant functions at $\{x, t\}$. We expand the classification of lifted envelope points from three types in a smooth sweep to four types in a CSG sweep:

- *Smooth Contour*: $t \in (0, 1)$, $f(x, t) = 0$, $\exists i \in \alpha(x, t)$ s.t. $f'_i(x, t) = 0$.
- *Sharp Contour*: $t \in (0, 1)$, $f(x, t) = 0$, $\exists i, j \in \alpha(x, t)$ s.t. $f'_i(x, t)f'_j(x, t) < 0$.
- *Bottom cap*: $t = 0$, $f(x, t) = 0$, $\exists i \in \alpha(x, t)$ s.t. $f'_i(x, t) > 0$.

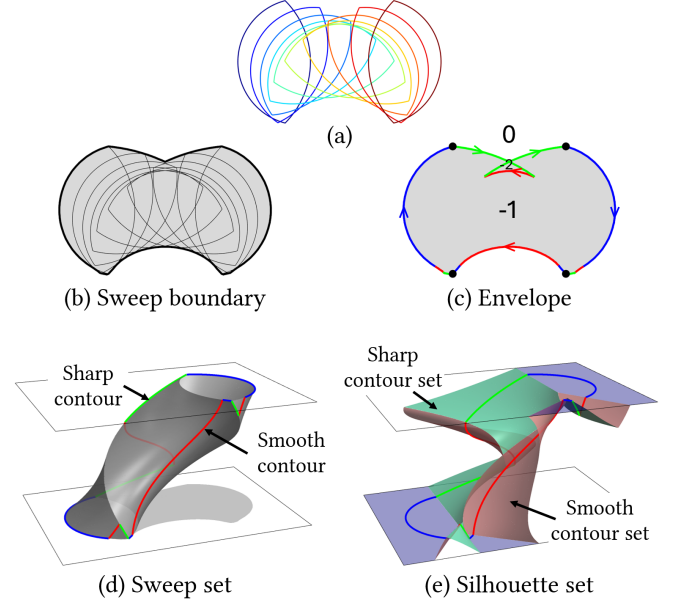


Fig. 5. A CSG sweep. (a) A rolling “lens” created by intersecting two moving disks. (b) The sweep volume and sweep boundary. (c) The oriented envelope (red, green, blue curves projected from smooth contours, sharp contours, and caps) and the winding number of each arrangement cell. (d) The 3D sweep set (gray surface) and the lifted envelope (red, green, and blue curves). (e) The 3D silhouette set (red, green, and blue surfaces).

- *Top cap*: $t = 1$, $f(x, t) = 0$, $\exists i \in \alpha(x, t)$ s.t. $f'_i(x, t) < 0$.

The smooth contour generalizes the *contour* type in [Ju et al. 2025] and lies on a smooth patch of the sweep set. The sharp contour is unique to a CSG sweep and lies on the sharp creases of the sweep set. These two types are shown as red and green curves in Figure 5 (d). The smooth and sharp contours project to parts of the envelope that are swept respectively by the smooth boundary patches and sharp features of the shape, as shown in Figure 5 (c). By genericity, each type of lifted envelope point forms an $(n - 1)$ -manifold in $\mathbb{R}^{n+1}$, and different types meet at $(n - 2)$ -manifolds.

Silhouette set. As in a smooth sweep, the lifted envelope in a CSG sweep can also be defined as the intersection between two implicit surfaces in $\mathbb{R}^{n+1}$. We define the *silhouette function* g as the time derivative of the dominant function with constant values on the top and bottom time planes:

$$g(x, t) = \begin{cases} f'_i(x, t), & 0 < t < 1, \forall i \in \alpha(x, t) \\ -1, & t = 0 \\ 1, & t = 1 \end{cases} \quad (2)$$

Since g is discontinuous, both at the top and bottom time planes and where there are multiple dominant functions, we take the boundary of the sublevel set $g(x, t) < 0$ as its zero-level set, the *silhouette set*. The lifted envelope is therefore the intersection of two zero-level sets: the sweep set (of the sweep function f) and the silhouette set (of the silhouette function g).

Like the sweep set, the silhouette set is also piecewise-smooth. It can be decomposed into four subsets, each containing one type of lifted envelope points (see Figure 5 (e)):

- *Smooth contour set*: $t \in (0, 1)$, $\exists i \in \alpha(x, t)$ s.t. $f'_i(x, t) = 0$.
- *Sharp contour set*: $t \in (0, 1)$, $\exists i, j \in \alpha(x, t)$ s.t. $f'_i(x, t)f'_j(x, t) < 0$.
- *Bottom cap set*: $t = 0$, $\exists i \in \alpha(x, t)$ s.t. $f'_i(x, t) > 0$.
- *Top cap set*: $t = 1$, $\exists i \in \alpha(x, t)$ s.t. $f'_i(x, t) < 0$.

Points on the silhouette set with the same set of dominant functions form a smooth n -manifold. In particular, points on the smooth contour set dominated by f_i lie on the zero-level set of f'_i , while points on the sharp contour set dominated by a pair $\{f_i, f_j\}$ lie on the zero-level set of the *difference* $f_i - f_j$.

Summary. To recap, the key difference between lifting a smooth sweep and a CSG sweep is that the latter introduces a new component to the lifted envelope, the *sharp contour*. Intuitively, the projection of the sharp contour is swept by the sharp features of the shape. The lifted envelope is an intersection of two implicit surfaces, which are both smooth in a smooth sweep but piecewise-smooth in a CSG sweep. For the latter, one of these two implicit surfaces (the sweep set) is made up of zero-level sets of member functions, and the other (the silhouette set) is made up of zero-level sets of the time derivatives and pairwise differences of member functions.

4.3 Characterizing sweep boundary

Ju et al. [2025] developed a robust and efficient characterization of the sweep boundary as a subset of the (projected) envelope. The key to their characterization is defining an *outward* orientation of the lifted envelope, which yields winding numbers over the arrangement of the envelope. Their orientation definition makes use of derivatives of the (smooth) sweep function and silhouette function. We will extend their orientation definition to piecewise-smooth sweep and silhouette functions, and show that the resulting winding numbers correctly identify the sweep boundary.

In a smooth sweep, the outward orientation at a point p on the lifted envelope is represented as a tangent basis Σ such that $\det(\{\Sigma, \nabla f(p), \nabla g(p)\}) > 0$, where f, g are the sweep and silhouette functions and ∇ denotes the gradient. This definition is readily applicable to the smooth contour and caps in a CSG sweep, where f is identical to one dominant member function and is smooth. However, it would not work on the sharp contour, where two dominant functions meet with C^0 continuity.

We orient a sharp contour by examining the gradients of its two dominant functions. Consider a point p on a sharp contour dominated by member functions f_1, f_2 such that $f'_1(p)f'_2(p) < 0$. Without loss of generality, we assume that $f'_1(p) > 0$ and $f'_2(p) < 0$. As the sharp contour lies on the zero-level sets of both f_1 and f_2 , a tangent basis Σ at p is orthogonal to both $\nabla f_1(p)$ and $\nabla f_2(p)$. Due to genericity, the two gradients are not parallel, and hence $\{\Sigma, \nabla f_1(p), \nabla f_2(p)\}$ forms a basis of $\mathbb{R}^{n+1}$. On the other hand, consider a binary-tree representation of the CSG expression associated with f , where the internal nodes are *min* and *max* operators and the leaves are the member functions. Let $\square$ be the operator at the unique internal node where the leaves representing f_1, f_2 reside in different child sub-trees. We say Σ is *outward* if either:

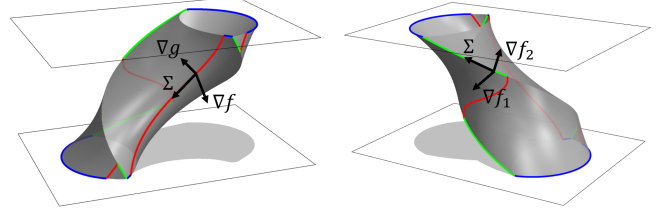


Fig. 6. Outward orientation on a smooth (left) and sharp (right) contour in the sweep example of Figure 5.

- $\square = \min$ and $\det(\{\Sigma, \nabla f_1(p), \nabla f_2(p)\}) > 0$, or
- $\square = \max$ and $\det(\{\Sigma, \nabla f_1(p), \nabla f_2(p)\}) < 0$.

Figure 6 illustrates the outward orientation assignment on the smooth and sharp contour of the CSG sweep in Figure 5.

To identify the sweep boundary, we first project the tangent basis from $\mathbb{R}^{n+1}$ (lifted envelope) to $\mathbb{R}^n$ (envelope). A point x on the envelope is said to be *regular* if it avoids the intersections and singularities. That is, there is a unique time t_x such that $\{x, t_x\}$ is on the smooth part of the lifted envelope, whose tangent space at $\{x, t_x\}$ does not include the time direction. By genericity, the non-regular points are isolated in the sense that their dimensionality is at most $n - 2$ (while the envelope is of dimension $n - 1$). The envelope is said to be *outward-oriented* if each regular point is associated with a tangent basis in $\mathbb{R}^n$ that is projected from an outward tangent basis of the lifted envelope at $\{x, t_x\}$. The sweep boundary can be identified as follows,

THEOREM 4.1. *The boundary of a CSG sweep in $\mathbb{R}^n$ is the subset of the outward-oriented envelope that bounds the subsets of $\mathbb{R}^n$ with winding number 0.*

The proof is given in the Supplemental Materials (Section 2.1). The proof also shows that an intuitive construction of the outward orientation in a smooth sweep, described in [Ju et al. 2025], works on a CSG sweep as well. Recall that the lifted envelope is the intersection of the zero-level sets of f and g . To obtain the outward orientation, first orient the silhouette set (zero-level set of g) so that it faces the subspace of $\mathbb{R}^{n+1}$ with positive g , then orient the lifted envelope (zero-level set of f on the silhouette set) so that it faces the subspace of the oriented silhouette set with positive f . Our algorithm leverages this construction to ensure the outward orientation of the discretized envelope (see Section 5).

4.4 Sharp features

We conclude this section with an analysis of the sharp features on the sweep boundary. Such features come in three flavors: self-intersections of the envelope, edges on the shape at the beginning or end of the sweep (called *sweep-end* features), and creases created during sweeping (called *in-sweep* features). The three types are colored green, red, and blue in Figure 2 (d).

Furthermore, we can characterize the sweep-end and in-sweep features by the structure of the lifted envelope. Consider a segmentation of the lifted envelope by the component type, namely the sharp or smooth contours and the top or bottom caps, and further by the set of dominant functions (see Figure 7 (a,b,c)). As explained

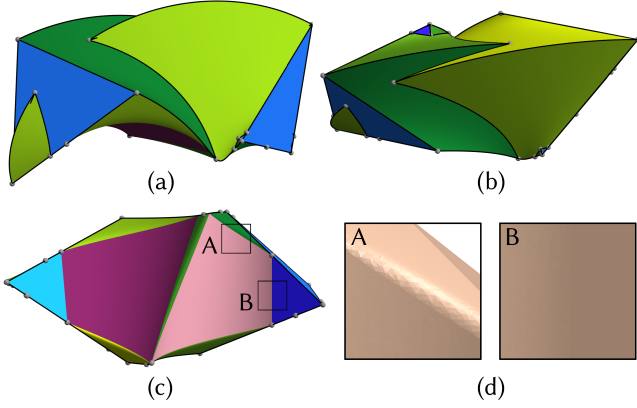


Fig. 7. The sweep boundary in Figure 2 (d) segmented by the component type of the (lifted) envelope and dominant function sets in three different views (a,b,c). Green-ish, red-ish, and blue-ish colors indicate sharp contour segments, smooth contour segments, and cap segments. The zoom-ins in (d) show there is no sharp edge between a smooth contour segment and adjacent segments.

in detail in the Supplemental Materials (Section 3), sweep-end features are located between two cap segments with different dominant functions or between a cap segment and a sharp contour segment, while in-sweep features are located between two sharp contour segments with different dominant functions. Note that a smooth contour segment always meets adjacent segments (contours or caps) with G^1 continuity (Figure 7 (d)).

5 Method overview

We now describe our method for discretely approximating a CSG sweep boundary in 3D. Our method follows the overall pipeline of [Ju et al. 2025], which first discretizes the envelope and then trims it to the sweep boundary. The process consists of four main steps and is illustrated in Figure 8 using a 2D sweep (Figure 5):

- (1) Generate a grid in 4D with an adaptive resolution so that the space around the lifted envelope is more refined.
- (2) Compute the silhouette set as a polyhedral mesh by contouring the silhouette function g in the grid.
- (3) Compute the lifted envelope as a triangular mesh by contouring the sweep function f on the silhouette set.
- (4) Compute the arrangement of the envelope and output triangles that bound arrangement cells with zero winding.

The pipeline produces a water-tight, intersection-free triangular mesh with embedded sharp features. Both sweep-end and in-sweep features are obtained from the dominant functions provided by the computation of the silhouette set (Step 2), and the self-intersections of the envelope are obtained by mesh arrangement (Step 4). The outward orientation of the lifted envelope required for trimming is obtained by orienting the silhouette set towards the 4D space with positive g in Step 2 and orienting the lifted envelope towards the 3D sub-space of the silhouette set with positive f in Step 3.

Our method leverages existing solutions for the last two steps and introduces new algorithms for the first two steps. The oriented

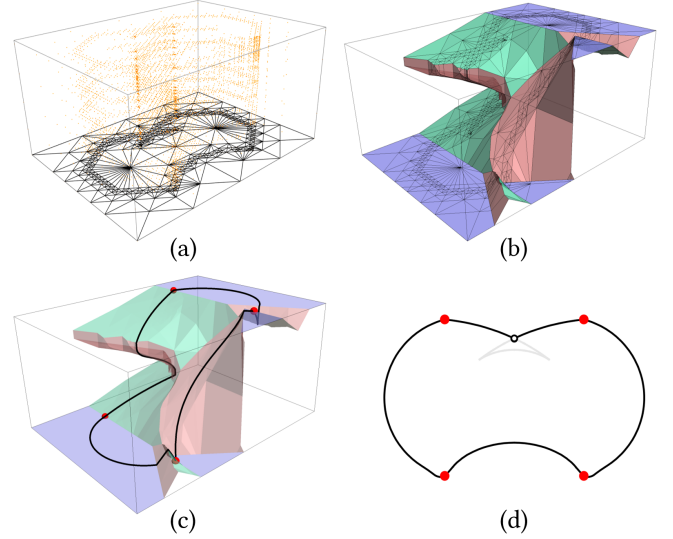


Fig. 8. Overview of our method on a 2D sweep (Figure 5). (a): Generating an adaptive grid consisting of a triangular mesh and time samples (orange) (Section 7). (b): Extracting the polygonal silhouette set on the grid (Section 6). (c): Computing the lifted envelope curve on the silhouette set. (d): Trimming the projected envelope.

lifted envelope in Step 3 is computed using the "Marching Polyhedra" algorithm in [Ju et al. 2025], and the mesh arrangement and the winding numbers in Step 4 are obtained from [Zhou et al. 2016]. Our main contribution is computing a piecewise-smooth silhouette set in Step 2, which we describe next (Section 6). We then present an extension of the 4D grid refinement algorithm in [Ju et al. 2025] to generate the adaptive grid in Step 1 (Section 7).

6 Computing the silhouette set

The silhouette set is the zero-level set of the silhouette function g (Equation 2) in the lifted space. One could approximate the silhouette set by directly contouring g using implicit surfacing methods designed for higher dimensions [Bhaniramka et al. 2004; Castelo et al. 2022; Schwaha and Heinzl 2010]. However, these methods cannot capture the sharp features of the piecewise-smooth silhouette set. This would result in the loss of sharp features on the lifted envelope, which is the zero-level set of the sweep function f on the silhouette set, and in turn, the loss of sharp features on the sweep boundary. To recover the sharp features, we shall explicitly compute the intersections between the implicit surfaces underlying each smooth component of the silhouette set.

We first introduce a general and robust algorithm for discretizing the *arrangement* of level sets of multiple functions – a full spatial decomposition of $\mathbb{R}^n$ that includes the level sets and their intersections (Section 6.1). We then adapt the algorithm to the pairwise *differences* between functions (Section 6.2), whose level sets contain the sharp contour set. Finally, we present the complete algorithm for computing the silhouette set (Section 6.3). All algorithms presented in this section are applicable to arbitrary dimensions.

6.1 Arrangement of implicit surfaces

We describe an algorithm that discretizes the arrangement of the zero-level sets of scalar functions $\{h_1, \dots, h_m\}$ in $\mathbb{R}^n$. Our algorithm works on any “black-box” functions where the only information available is the function values at a query point, and it does not require h_i to be generic.

We follow the iterative refinement approach taken by existing arrangement methods designed for 3D functions [Bagley et al. 2016; Du et al. 2022; Guo et al. 2021; Kim et al. 2000]. An initial grid is sequentially “cut” by the zero-level sets of $h_1, \dots, h_m$, where each cut results in a more refined grid that embeds the zero-level set as a subset of grid elements. We represent the grid as a *cell complex*, a collection of cells of different dimensions that decompose a domain $\Omega \subset \mathbb{R}^n$. A k -dimensional cell (or k -cell) is a k -dimensional polytope, which is in turn bounded by $(k - 1)$ -cells (if $k > 0$) called *faces*. The cell complex representation offers greater flexibility than the commonly used simplicial grids (e.g., tetrahedral meshes in 3D). It not only accommodates a wider range of grid types, such as the columnar grid used in [Ju et al. 2025] and our method (see Section 7), but also avoids the need to re-tessellate cells after each cut.

Cutting a cell complex C by the zero-level set of h_i generates a new cell complex, where each cell is associated with a sign of h_i . The zero-level set of h_i consists of all zero-signed cells. Our algorithm processes cells of C in the order of increasing dimensionality. A k -cell whose vertices exhibit a sign change is split by one or more *walls*, which are $(k - 1)$ -cells with a zero sign, into two or more *rooms*, which are k -cells with positive or negative signs. The rooms and walls in a cell are constructed, in turn, from rooms and walls on the faces of the cell. The algorithm proceeds in the following steps:

- (1) For each 0-cell c (a vertex), evaluate $h_i(c)$. Exact zeros are replaced by a small positive constant.
- (2) For each 1-cell c (an edge) connecting two vertices with different signs, create a new vertex (a *0-wall*) with a zero sign at the zero-crossing on c computed by linear interpolation. The new vertex splits c into two edges (1-rooms), each inheriting the sign of a vertex of c (Figure 9 (a)).
- (3) For each 2-cell c (a polygon) where at least one boundary edge has been split, traverse the boundary and connect successive pairs of 0-walls into new edges (1-walls) with a zero sign. The new edges split c into multiple polygons (2-rooms), each with a positive (resp. negative) sign and bounded by a connected component of the non-negative (resp. non-positive) set of c 's faces, 1-rooms on the faces, and the newly built 1-walls (Figure 9 (b)).
- (4) For each $k = 3, \dots, n$ and each k -cell c where at least one face has been split, create one new $(k - 1)$ -cell (a $(k - 1)$ -wall) with a zero sign for each connected component of $(k - 2)$ -walls on the faces. These walls split c into multiple k -cells (k -rooms), each with a positive (resp. negative) sign and bounded by a connected component of the non-negative (resp. non-positive) set of c 's faces, $(k - 1)$ -rooms on the faces, and the newly built $(k - 1)$ -walls (Figure 9 (c)).

The result of sequential cutting is a cell complex where each cell is associated with m signs, one for each function. The discrete zero-level set of h_i consists of all cells having a zero sign for h_i ,

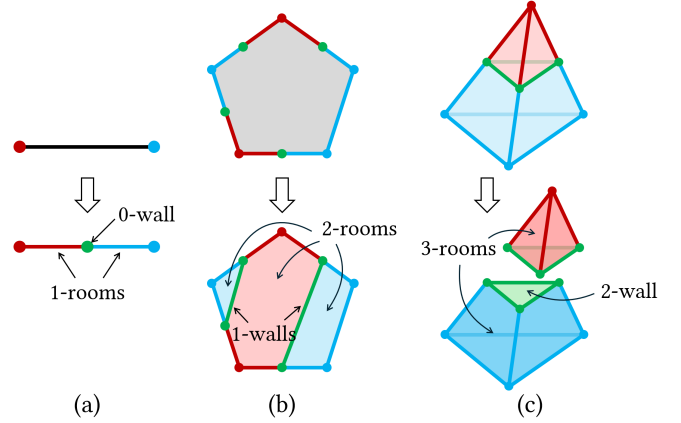


Fig. 9. Splitting a 1-cell (a), 2-cell (b), and 3-cell (c). Positive, negative and zero-signed cells are colored red, blue, and green, respectively.

and the intersection of two or more zero-level sets consists of cells having a zero sign for each function. We show that our algorithm discretizes each level set, or intersection between level sets, as a gap-free manifold of the appropriate dimension (∂C denotes the boundary of C):

THEOREM 6.1. *The discrete intersection of zero-level sets of k functions $H \subseteq \{h_1, \dots, h_m\}$, for any $k = 1, \dots, \min(m, n)$, computed by the arrangement algorithm on an n -manifold cell complex C is a (possibly empty) $(n - k)$ -manifold whose boundary is on ∂C .*

The proof is given in the Supplemental Materials (Section 2.2). Our cell complex representation can be enriched with the assignment of cell *orientations*, which are important later on for orienting the silhouette set. The orientation of a cell is expressed by the orientation of its faces. For example, the orientation of a polyhedron in 3D can be defined by the orientation of its polygonal faces (i.e., inward or outward), and each polygon's orientation, in turn, can be defined by the edge directions. An orientation assignment is *consistent* if all faces of each cell are oriented inward or outward, and if faces shared by adjacent n -cells are assigned opposite orientations. In the Supplemental Materials (Section 4), we detail the definition of orientation and conditions of consistency, and we show how our arrangement algorithm can be augmented to produce a consistently oriented cell complex.

6.2 Dominance map

The arrangement algorithm presented above ensures that the intersections of the discrete level sets have the proper dimensions. However, the level sets of difference functions $f_i - f_j$, which we call the *equal set* of $\{f_i, f_j\}$, violate this property: the intersection of equal sets of $\{f_i, f_2\}$ and $\{f_2, f_3\}$ also lies on the equal set of $\{f_i, f_3\}$, making the intersection of the three equal sets have the same dimension as the intersection of two equal sets. As our algorithm enforces the proper dimensions of the intersections, it would fail to identify the common intersections among the three equal sets, thus producing cells with inconsistent orderings of functions (see the red triangle in Figure 10).

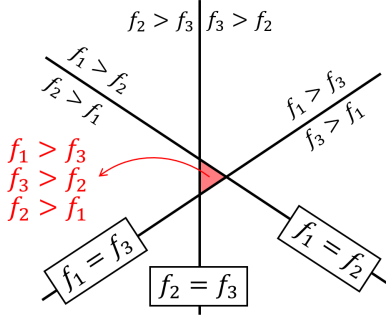


Fig. 10. Discrete arrangement of equal sets among three functions f_1, f_2, f_3 may produce an inconsistent ordering (red triangle).

Instead of modifying the arrangement algorithm to work on the equal sets of difference functions, which may involve a non-trivial effort, our solution is to compute the arrangement of a different group of equal sets. This new group contains the sharp contour set, just like the equal sets of the difference functions, but has intersections at proper dimensions. Consider a sweep function f in $\mathbb{R}^{n+1}$ defined by a CSG expression and member functions $f_1, \dots, f_m$. As in Section 4.3, we consider the CSG expression as a binary tree, T , with the member functions at the leaves and the *min*/*max* operators at the internal nodes. Note that the subtree rooted at an internal node defines a “partial” CSG function, which we call a *node function*. Unlike the smooth member functions at the leaves, the node functions at the internal nodes are only piecewise-smooth. The node function at the root of T is identical to the sweep function f .

Our key observation is that the equal sets between the (node or member) functions at the two children of each internal node intersect at proper dimensions. Denote the $m - 1$ internal nodes of T as $v_1, \dots, v_{m-1}$, and let S_i be the equal set of v_i ’s two child (node or member) functions. We prove in the Supplemental Materials (Section 2.3) that:

THEOREM 6.2. *Let f be a CSG sweep function in $\mathbb{R}^{n+1}$ with m member functions.*

- (1) *The union $\bigcup_i S_i$ contains the sharp contour set of f .*
- (2) *For any subset of k equal sets $S \subseteq \{S_1, \dots, S_{m-1}\}$, where $k \in [1, \min(n+1, m-1)]$, $\bigcap S$ is an $(n - k + 1)$ -manifold.*

Intuitively, the equal set S_i consists of points where the member functions *dominant* in the node function r_i at node v_i are the union of those dominant in the two child functions of v_i . The equal set divides the space into two regions, each consisting of points where the dominant member functions in r_i come from the same child node of v_i . As a result, the union of all S_i decomposes $\mathbb{R}^{n+1}$ into regions within which all points share the same dominant member functions at *each* internal node of T . We therefore call the arrangement of S_i the *dominance map*.

The dominance map can be discretized by applying our arrangement algorithm to the $m - 1$ differences between the child (member or node) functions at every internal node. Since each node function r_i is determined from the functions at the children of v_i , we order the difference functions in the arrangement computation by a *pre-order*

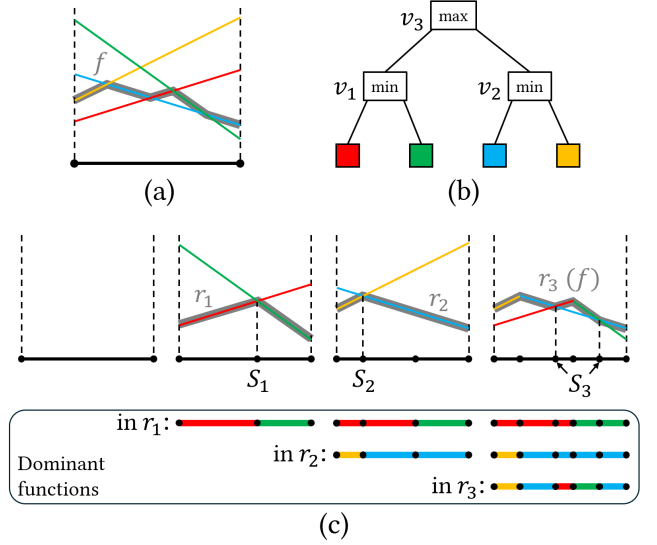


Fig. 11. Computing a 1D dominance map of four linear functions (a) and a CSG tree (b) by iteratively cutting a cell complex (a single 1-cell) with the equal sets S_i of the tree nodes v_i for $i = 1, 2, 3$ (c). Each step in (c) shows the two child functions (thin colored lines) that define the equal set S_i and the parent node function r_i (thick gray lines), as well as the dominant member function (indicated by color) in each r_i at each 1-cell (bottom).

of the internal nodes (i.e., parents are preceded by their children). For the purpose of extracting the silhouette set, we also record the dominant member function(s) in r_i at each cell after cutting by the equal set S_i . Figure 11 shows an example of computing the dominance map of four 1-dimensional (linear) member functions (a) and a CSG tree (b), showing the refined 1-dimensional cell complex and updated dominant functions after each cut (c).

6.3 Silhouette set

We now describe our algorithm for computing the silhouette set of a sweep in $\mathbb{R}^n$ defined by a CSG sweep function f in $\mathbb{R}^{n+1}$. Recall that the silhouette set consists of the top and bottom cap sets, the smooth contour set, and the sharp contour set.

We first compute a cell complex where components of the silhouette set are embedded as cells. We start with an initial $(n + 1)$ -dimensional cell complex C where the top and bottom time planes ($t = 0, 1$) are embedded as subsets of cells (see the next section). As a result, C contains the top and bottom cap sets. Next, we compute the arrangement of the zero-level sets of f'_t on C (Section 6.1), producing a refined complex C' that contains the smooth contour set. Finally, we compute the dominance map on C' (Section 6.2), producing the final complex C^* that contains the sharp contour set. These steps are illustrated in Figure 12 (a,b,c) for the 2D example in Figure 5.

Next, we extract the silhouette set from the refined complex C^* (Figure 12 (d)). The arrangement algorithm equips each cell with the dominant functions and the time derivative signs of all member functions. We call the sign of the time derivative of the dominant function at each $(n + 1)$ -cell its *time sign*. Following the definition

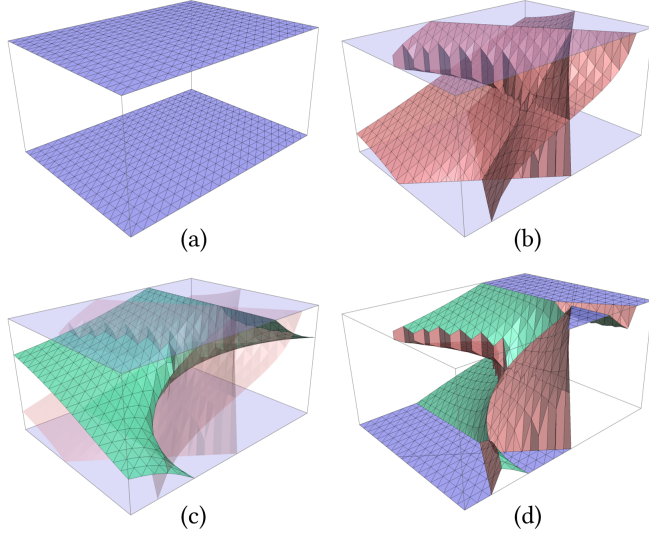


Fig. 12. Computing the silhouette set: initial cell complex (a), computing the arrangement of zero-level sets of time derivatives (b), computing the dominance map (c), and extracting components of the silhouette set (d).

of the silhouette set components in Section 4.2, we can identify the n -cells in C^* that make up each component as:

- Smooth (resp. sharp) contour set: n -cells with one (resp. two) dominant function(s) and are faces of two $(n + 1)$ -cells with different time signs.
- Bottom (resp. top) cap set: n -cells on the bottom (resp. top) time plane that are faces of a single $(n + 1)$ -cell with positive (resp. negative) time sign.

We orient each n -cell towards (resp. away from) the incident $(n + 1)$ -cell with positive (resp. negative) time sign, using the oriented cell complex representation described in the Supplemental Materials (Section 4). We also pass the component type and dominant functions in each n -cell of the silhouette set onto the $(n - 1)$ -cells of the lifted envelope computed within them, and further onto the projected and trimmed envelope. These attributes enable the extraction of sharp features on the sweep boundary, as characterized in Section 4.3.

7 Grid generation

To compute the lifted envelope of a sweep in $\mathbb{R}^n$, we need a grid that discretizes the spatial-temporal domain in $\mathbb{R}^{n+1}$ that encompasses the lifted sweep volume. To capture the possibly complex geometry of the lifted envelope without an excessively large number of elements, it is essential that the grid has a varying resolution in space and time that *adapts* to the geometry of the lifted envelope.

Our grid generation method for $n = 3$ closely follows that of [Ju et al. 2025]. Their method generates a *columnar* grid that consists of a tetrahedral mesh in 3D and *time samples* at each mesh vertex. The 4D grid is generated by extruding each tetrahedron in time, creating a 4D “column” whose vertices are the time samples at the vertices of the tetrahedron. This “3.5-D” representation has several benefits for computing the sweep surface. Compared with a simplicial grid,

the columnar grid has fewer grid elements, which leads to faster computation of the lifted envelope and fewer triangles on the sweep surface. It also tends to result in better-shaped triangles on the sweep surface. Last but not least, generating a columnar grid with adaptive spatial-temporal resolution can leverage existing methods for simplicial refinement in 3D (e.g., [Ju et al. 2024]).

We adopt the columnar grid representation and modify the generation method in [Ju et al. 2025]. Given an initial grid, their method refines the columns either in space (by refining the tetrahedral mesh) or in time (by adding time samples to mesh vertices). A 4D column c is considered for refinement if it meets the following criteria:

- (1) c is not completely inside the lifted sweep volume.
- (2) c intersects with the lifted envelope.
- (3) The discretization error of the silhouette set (or lifted envelope) in c exceeds some threshold ϵ_{sil} (or ϵ_{env}).

These criteria are implemented on a simplicial decomposition of c whose vertices are the time samples. Checking each criterion boils down to either testing for zero-crossing (1,2), or bounding the level set approximation error (3), of a single function or a pair of functions in a 4D simplex.

We follow the same set of criteria, but extend their implementation to a CSG sweep. While both the silhouette set and the lifted envelope in a smooth sweep are defined by a single function, they are defined by multiple functions in a CSG sweep. In particular, the smooth and sharp contour set on the silhouette set is locally defined by one or two functions that are *dominant*, and the same is for the smooth and sharp contour on the lifted envelope. To check the refinement criteria in a 4D simplex s , we first identify the set D of dominant functions as the union of dominant functions at the vertices of s . We then check the criteria in s for the smooth contour (set) defined by *each* function in D and for the sharp contour (set) defined by *each pair* of functions in D . An example grid for the 2D sweep in Figure 5 is shown in Figure 8 (a).

8 Results

We evaluate our method on both rigid and non-rigid sweeps in 3D. Our algorithm is implemented in C++, and the experiments are conducted on a Windows PC with an AMD 3970x CPU and 256 GB of RAM. All inputs are scaled to a unit bounding box. The only parameters are those controlling the grid refinement. The initial tetrahedral grid is generated by Kuhn’s subdivision of a 4^3 cubical grid of the bounding box. Unless stated otherwise, the refinement parameters $\epsilon_{sil}, \epsilon_{env}$ (see Section 7) are both set to be 10^{-4} .

8.1 Validation

To validate the accuracy of the sweep surface and feature curves produced by our method, we consider a tetrahedron moving and rotating around a common axis (i.e., a screw motion); see Figure 13 (a,b). The sweep boundary, which is identical to the envelope, has a simple analytical form consisting of two triangles at each end of the sweep connected by four ruled surfaces, each swept by a tetrahedral edge. The triangles and ruled surfaces are separated by sharp features that are either straight or spiral segments.

We measure the Chamfer Distance (CD) of the sweep surface and feature curves computed by our method with a fine tessellation of

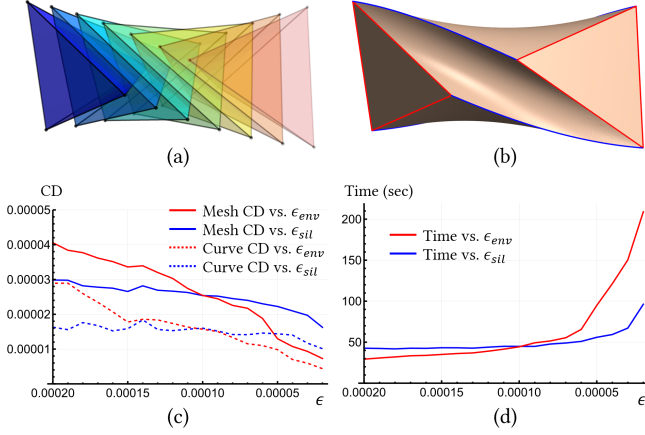


Fig. 13. (a) A tetrahedron under screw motion. (b) Sweep surface with feature curves. (c) Chamfer distance between the sweep surface and feature curves computed by our method and ground truth with decreasing ϵ_{env} , ϵ_{sil} . (d) Running time as ϵ_{env} , ϵ_{sil} decrease.

their analytical counterparts. Using our default parameter setting ($\epsilon_{sil} = \epsilon_{env} = 10^{-4}$), the surface and curve CDs are respectively 2.5×10^{-5} and 1.6×10^{-5} . We also plot the CDs as each parameter varies while keeping the other parameter at its default value in Figure 13 (c). Observe that the errors converge to zero at a near-linear rate with the decreasing ϵ_{env} , the parameter that directly controls the approximation accuracy of the envelope, while decreasing ϵ_{sil} has a less direct impact on errors.

8.2 Comparisons

We compare our method with two recent sweep surfacing methods [Ju et al. 2025; Sellán et al. 2021]. Like our method, Ju et al. [2025] can sweep deformable shapes, but it is restricted to smooth functions. To test it on a piecewise-smooth CSG function, we replace the min/max operations with “soft” booleans (see details in Supplemental Materials Section 5). The implicit continuation method of Sellán et al. [2021] works on any shape represented as a signed distance function (SDF), including piecewise-smooth surfaces such as a triangular mesh. However, it is limited to rigid-body motions. As a result, we use only rigid sweeps for this comparison. While [Adsul et al. 2016; Machchhar et al. 2017] can also sweep piecewise-smooth shapes under rigid-body motions, we are not able to obtain their code or program for comparison.

We conduct the comparison on two rigid sweeps, a rolling tetrahedron (Figure 2) and a moving and twisting cube (Figure 16). In both examples, the method of Sellán et al. does not produce any sharp features. While Ju et al. can identify sharp creases where the envelope self-intersects (green curves), other sharp features are rounded due to the use of soft boolean to create a smooth function. Our method identifies both the in-sweep (blue curves) and sweep-end (red curves) features. Observe from the inserts that the meshes produced by our method are much sharper at those features than other methods at a comparable mesh size.

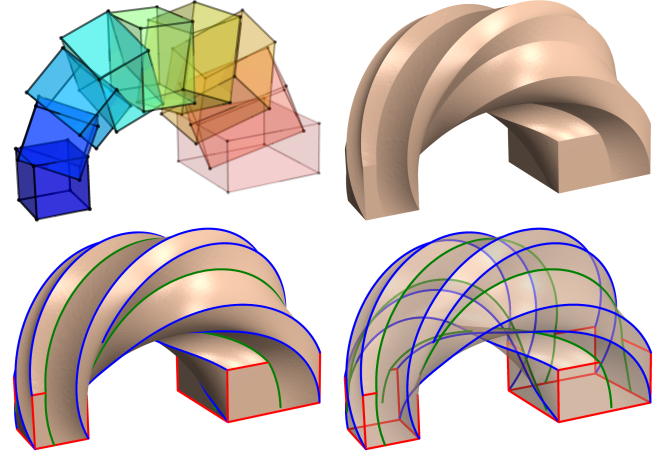


Fig. 14. Sweep surface of a moving, twisting, and stretching cube (top-left).

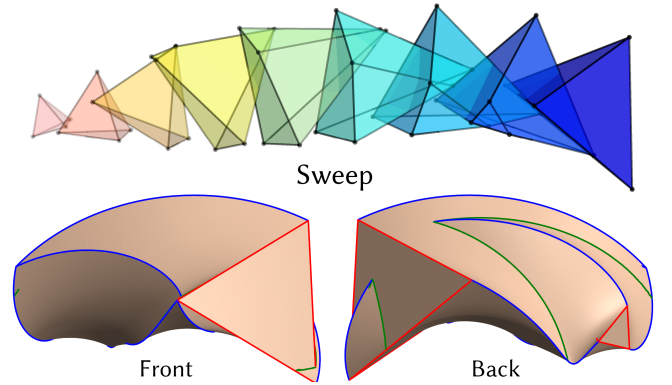


Fig. 15. Sweep surface (bottom, showing two sides) of a rolling tetrahedron cut by a moving plane (top, the trajectory is stretched out for visualization).

8.3 Deforming sweeps

We next demonstrate our method’s ability to sweep deforming shapes while capturing the sharp features. Figure 14 modifies the twisting cube example of Figure 16 by stretching the cube in two directions as it moves, creating a sweep surface that shares a similar structure of feature curves with that in Figure 16. A larger deformation is seen in Figure 15, where a tetrahedron is “shaved” by a moving plane (from far to near) as the tetrahedron rolls, thus changing its shape from a tetrahedron to a triangular prism and back to a tetrahedron. An example involving a CAD model of a bolt is shown in Figure 17, where the shaft shortens as the bolt spins. The bottom picture visualizes the composition of the sweep surface as sharp contour segments (greenish patches), smooth contour segments (reddish patches), and cap segments (blueish patches).

While the sweeps above involve only changes in the shape’s geometry, our method can handle topological changes as well. Figure 18 shows a rolling tetrahedron being subtracted by a rolling torus that moves in the opposite direction. Note that the tetrahedron breaks into multiple components in the middle of the sweep. A

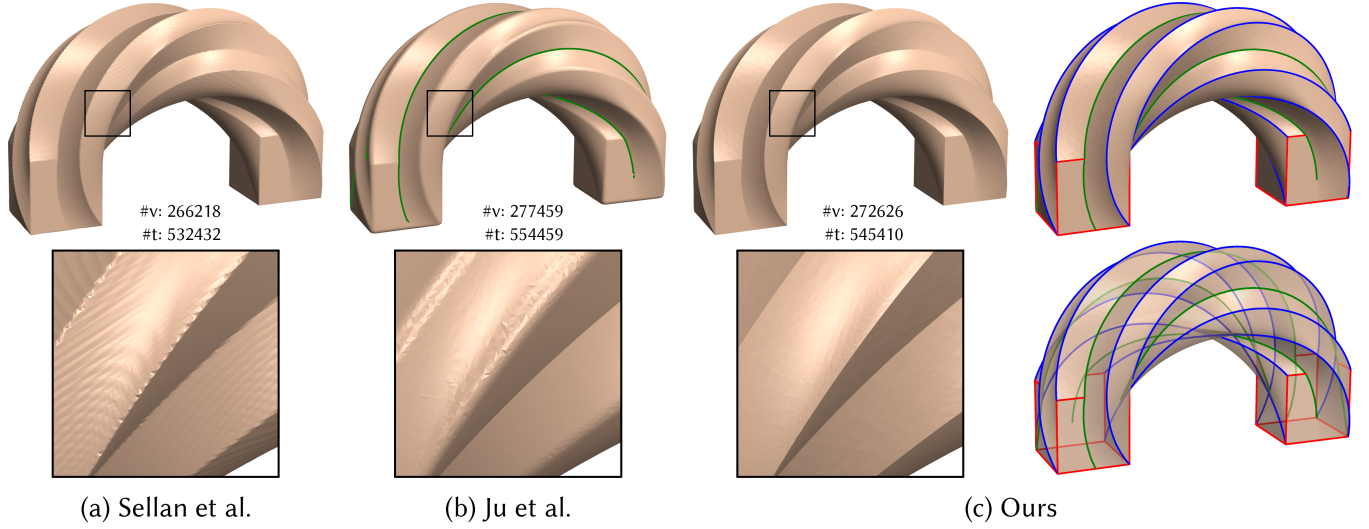


Fig. 16. Comparing [Sellán et al. 2021] (a), [Ju et al. 2025] (b), and ours (c, with different surface opacity and with or without curves) on a moving and twisting cube. In-sweep features, sweep-end features, and envelope self-intersections are marked by blue, red, and green curves.

larger topological change is demonstrated in Figure 1, where a growing sphere is subtracted from a rolling tetrahedron, altering both its genus (from 3 to 0) and number of components (from 1 to 4). Finally, Figure 19 shows two sweep sequences, both involving a rolling genus-5 wire ball defined by subtracting 3 cylinders and a sphere from a larger sphere. In addition, the shape in the first sweep is cut by a plane that moves downward, changing its genus from 5 to 0. The shape in the second sweep is cut by two moving planes into a thin moving slice made up of a varying number of components. To better capture the complex surface geometry, we used a smaller parameter setting of $\epsilon_{env} = \epsilon_{sil} = 5 \times 10^{-5}$ for these two sweeps.

In all examples, our method is able to model the complex network of feature curves on the sweep surface and keep the meshes sharp at those features.

8.4 Performance

The running time of our method is dominated by the computation of the lifted envelope, particularly the silhouette set in 4D (Section 6). Main factors affecting the performance include the grid resolution (controlled by the refinement parameters), the number of CSG member functions, and the geometric complexity of the silhouette set. Figure 13 (d) plots the runtime for the validation example as a function of the refinement parameters ϵ_{env} , ϵ_{sil} . We have observed a similar growth with the refinement level in other examples.

To evaluate the impact of the complexity of the CSG function on performance, we create two examples with a tunable number of member functions (see Figure 20 (a)), one resulting from intersecting linear half-spaces, and another from subtracting smaller spheres from a larger sphere. Both shapes are swept in a screw motion (same as in Figure 13 (a)). Observe in Figure 20 (b) that the running times in both examples exhibit a super-linear growth with the number of member functions. This is because computing the silhouette set involves $O(m)$ cuts on the cell complex for m member functions

Table 1. Statistics and timing (in seconds) for examples in the paper.

Fig.	m	tets	time samples	vertices	triangles	time (envelope)
15	5	338,424	1,933,760	159,489	318,974	167.21 (108.09)
2	4	393,616	2,498,760	188,103	376,202	188.84 (106.56)
18	5	482,853	2,448,003	274,576	549,194	272.8 (151.91)
1	5	516,496	1,504,572	216,682	433,372	315.96 (228.84)
14	6	411,301	4,476,480	247,177	494,394	492.49 (325.44)
16	6	451,301	4,479,625	272,626	545,410	618.71 (396.9)
19 (2)	7	436,335	2,241,639	298,649	597,358	996.99 (900.21)
19 (1)	6	900,823	5,517,615	630,567	1,261,459	2692.2 (2040.63)
17	10	747,875	11,202,685	763,527	1,527,326	4162.11 (3122.87)

(m cuts by the zero-level sets of the time derivative functions and $m - 1$ cuts to compute the dominance map), and the cost of each successive cut increases due to the increasing number of cells.

Finally, we report the performance of our method on the examples in the paper in Table 1, along with the number of member functions m , grid size (3D tetrahedra and time samples), and output mesh size (vertices and triangles). Observe that the runtime is dominated by envelope computation and generally grows with the grid size and number of member functions.

9 Discussion

We present a method for surfacing the sweep volume of a moving and deforming 3D piecewise-smooth solid. Our method generalizes the lifting formulation of sweeping from a single smooth function to multiple functions composed by a CSG expression. Leveraging a robust algorithm for computing multiple intersecting implicit surfaces in higher dimensions, our method is able to model the rich set of sharp features on the sweep boundary for general deformations.

Our work has a number of limitations. Due to the heuristic nature of our top-down refinement approach for generating the grid, our method cannot guarantee to approximate the sweep boundary within a distance bound or to produce a correct surface topology.

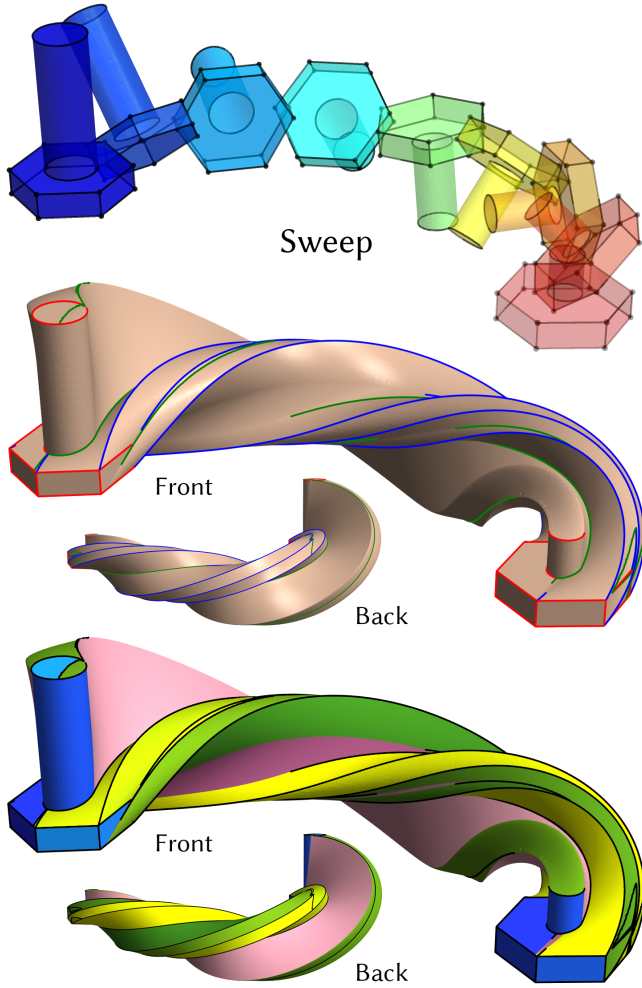


Fig. 17. Sweep surface of a moving bolt with a shortening shaft (top) shown in front and back views. The patches in the bottom are colored by component types of the envelope and the dominant function sets (see Figure 7).

The computation of the silhouette set in 4D is currently a key bottleneck in performance, and it can take a significant amount of time for complex sweep surfaces involving many CSG primitives (see Table 1). Potential acceleration strategies may include parallelization and reducing unnecessary cuts in the cell complex. It would also be interesting to explore how the lifting paradigm can be extended for sweeps along multiple dimensions (not just time), such as the joint angles in a multi-joint robot.

The genericity assumptions of our theoretical analysis - that the level sets of member functions (and derivatives) and their intersections should have proper dimensions - could be overly restrictive, as it excludes common scenarios such as stationary or identical member functions. While our algorithm enforces the proper dimensions by perturbation (in Step (1) in Section 6.1) and thus can handle non-generic inputs, it could lead to artifacts on the sweep surface. As an example, the shape in Figure 21 (a) is the union of a capped

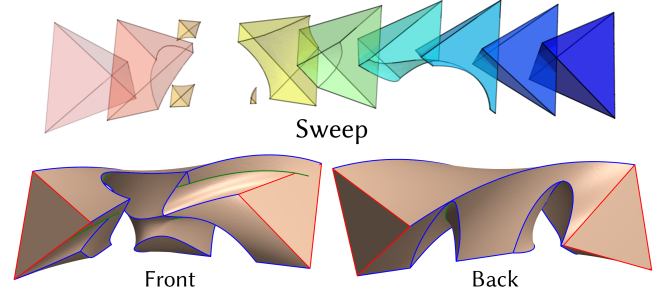


Fig. 18. Sweep surface of a rolling tetrahedron subtracted by a rolling torus in the opposite direction.

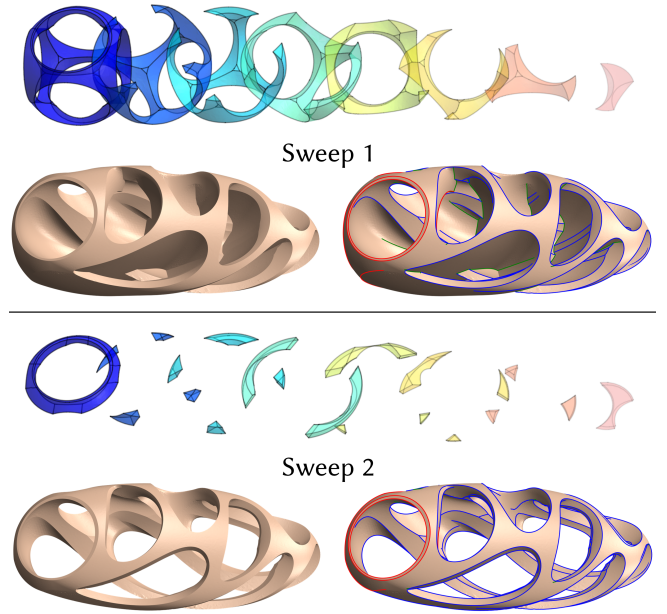


Fig. 19. Two sweeps with varying topology where a rolling genus-5 wire ball is cut by one (top) or two (bottom) moving planes.

cylinder with a hemisphere, where the top cylinder cap and the hemisphere base share the same plane (i.e., identical member functions). In addition, the shape translates in a direction parallel to that plane, and hence the plane stays unchanged during the sweep (i.e., a stationary function). Our method produces an erroneous interior sheet (arrow in (c)), since it considers the two identical planes (cylinder cap and hemisphere base) different by perturbation, and incorrectly labels the feature curves (arrow in (b)). Extending the lifted formulation to non-generic inputs and designing an algorithm that can properly handle such inputs without perturbation would be another interesting direction to explore.

Acknowledgments

This work is supported by NSF grant HCC-2401224 and a gift from Adobe Research.

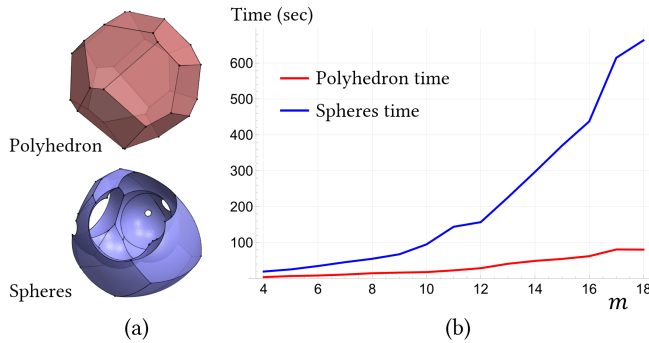


Fig. 20. (a): Shapes defined by a variable number m of planes (top) and spheres (bottom). (b) Running time for surfacing each shape swept in a screw motion (see Figure 13 (a)) as a function of m .

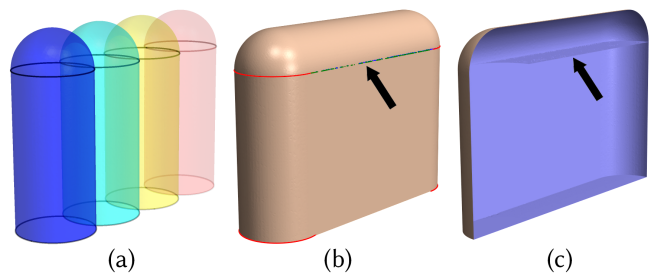


Fig. 21. A non-generic sweep (a), the sweep surface computed by our method (b), and a cross-sectional view (c).

References

- Karim Abdel-Malek, Jingzhou Yang, and Denis Blackmore. 2001. On swept volume formulations: implicit surfaces. *Computer-Aided Design* 33, 1 (2001), 113–121.
- Karim Abdel-Malek and Harn-Jou Yeh. 1997. Geometric representation of the swept volume using Jacobian rank-deficiency conditions. *Computer-Aided Design* 29, 6 (1997), 457–468.
- Steven Abrams and Peter K Allen. 1995. Swept volumes and their use in viewpoint computation in robot work-cells. In *Proceedings. IEEE International Symposium on Assembly and Task Planning*. IEEE, 188–193.
- Bharat Adsul, Jinesh Machchhar, and Milind Sohoni. 2015. A computational framework for boundary representation of solid sweeps. *Computer-Aided Design and Applications* 12, 2 (2015), 181–191.
- Bharat Adsul, Jinesh Machchhar, and Milind Sohoni. 2016. Incorporating sharp features in the general solid sweep framework. In *Computer Graphics Forum*, Vol. 35. Wiley Online Library, 21–31.
- Eugene L Allgower and Phillip H Schmidt. 1985. An algorithm for piecewise-linear approximation of an implicitly defined manifold. *SIAM journal on numerical analysis* 22, 2 (1985), 322–346.
- Brigham Bagley, Shankar P Sastry, and Ross T Whitaker. 2016. A marching-tetrahedra algorithm for feature-preserving meshing of piecewise-smooth implicit surfaces. *Procedia engineering* 163 (2016), 162–174.
- Praveen Bhaniramka, Rephael Wenger, and Roger Crawfis. 2004. Isosurface construction in any dimension using convex hulls. *IEEE Transactions on Visualization and Computer Graphics* 10, 2 (2004), 130–141.
- Denis Blackmore and Ming C Leu. 1992. Analysis of swept volume via Lie groups and differential equations. *The International Journal of Robotics Research* 11, 6 (1992), 516–537.
- Denis Blackmore, Ming C Leu, and Frank Shih. 1994. Analysis and modelling of deformed swept volumes. *Computer-Aided Design* 26, 4 (1994), 315–326.
- Denis Blackmore, Ming C Leu, and Liping P Wang. 1997. The sweep-envelope differential equation algorithm and its application to NC machining verification. *Computer-Aided Design* 29, 9 (1997), 629–637.
- Jean-Daniel Boissonnat, Siergry Kachanovich, and Mathijs Wintraecken. 2023. Tracing Isomanifolds in d in Time Polynomial in d using Coxeter–Freudenthal–Kuhn Triangulations. *SIAM J. Comput.* 52, 2 (2023), 452–486.
- Marcel Campen and Leif Kobbelt. 2010. Exact and robust (self-) intersections for polygonal meshes. In *Computer Graphics Forum*, Vol. 29. Wiley Online Library, 397–406.
- Antonio Castelo, Lucas Moutinho Bueno, and Marcio Gameiro. 2022. A combinatorial marching hypercubes algorithm. *Computers & Graphics* 102 (2022), 67–77.
- Jason Conkey and Kenneth I Joy. 2000. Using isosurface methods for visualizing the envelope of a swept trivariate solid. In *Proceedings the Eighth Pacific Conference on Computer Graphics and Applications*. IEEE, 272–280.
- Bruno Rodrigues De Araújo, Daniel S Lopes, Pauline Jepp, Joaquim A Jorge, and Brian Wyvill. 2015. A survey on implicit surface polygonization. *ACM Computing Surveys (CSUR)* 47, 4 (2015), 1–39.
- Xingyi Du, Qingnan Zhou, Nathan Carr, and Tao Ju. 2022. Robust computation of implicit surface networks for piecewise linear functions. *ACM Transactions on Graphics (TOG)* 41, 4 (2022), 1–16.
- Hüseyin Erdim and Horea T Ilies. 2008. Classifying points for sweeping solids. *Computer-Aided Design* 40, 9 (2008), 987–998.
- Hüseyin Erdim and Horea T Ilies. 2010. A comparison of sampling strategies for computing general sweeps. *Computer-Aided Design* 42, 8 (2010), 657–669.
- Jiateng Guo, Xulei Wang, Jiangmei Wang, Xinwei Dai, Lixin Wu, Chaoling Li, Fengdan Li, Shanjuan Liu, and Mark Walter Jessell. 2021. Three-dimensional geological modeling and spatial analysis from geotechnical borehole data using an implicit surface and marching tetrahedra algorithm. *Engineering Geology* 284 (2021), 106047.
- Horea T Ilies and Vadim Shapiro. 1999. The dual of sweep. *Computer-Aided Design* 31, 3 (1999), 185–201.
- Yiwen Ju, Xingyi Du, Qingnan Zhou, Nathan Carr, and Tao Ju. 2024. Adaptive grid generation for discretizing implicit complexes. *ACM Trans. Graph.* 43, 4, Article 82 (July 2024), 17 pages. doi:10.1145/3658215
- Yiwen Ju, Qingnan Zhou, Xingyi Du, Nathan Carr, and Tao Ju. 2025. Lifted Surfacing of Generalized Sweep Volumes. *ACM Transactions on Graphics (TOG)* 44, 6 (2025), 1–17.
- J Kieffer and FL Litvin. 1991. Swept volume determination and interference detection for moving 3-D solids. (1991).
- Dae Hyun Kim, Ulf Doering, and Beat Bruderlin. 2000. Polygonization of non-manifolds with the aid of interval operators. In *Proc. Implicit Surfaces*, Vol. 145. 151.
- Jinesh Machchhar, Denys Plakhotnik, and Gershon Elber. 2017. Precise algebraic-based swept volumes for arbitrary free-form shaped tools towards multi-axis CNC machining verification. *Computer-Aided Design* 90 (2017), 48–58.
- Claudia Madrigal and Ken Joy. 1999. Generating the envelope of a swept trivariate solid. (1999).
- Ralph R Martin and PC Stephenson. 1990. Sweeping of three-dimensional objects. *Computer-Aided Design* 22, 4 (1990), 223–234.
- Martin Peternell, Helmut Pottmann, Tibor Steiner, and Hongkai Zhao. 2005. Swept volumes. *Computer-Aided Design and Applications* 2, 5 (2005), 599–608.
- Margot Rabl, Bert Jüttler, and Laureano Gonzalez-Vega. 2008. Exact envelope computation for moving surfaces with quadratic support functions. *Advances in Robot Kinematics: Analysis and Design* (2008), 283–290.
- Jarek Rossignac, Jay J Kim, SC Song, KC Suh, and CB Jeong. 2007. Boundary of the volume swept by a free-form solid in screw motion. *Computer-Aided Design* 39, 9 (2007), 745–755.
- William J Schroeder, William E Lorensen, and Steve Linthicum. 1994. Implicit modeling of swept surfaces and volumes. In *Proceedings Visualization'94*. IEEE, 40–45.
- Philipp Schwaha and René Heinzl. 2010. Marching simplices. In *AIP conference proceedings*, Vol. 1281. American Institute of Physics, 1651–1654.
- Silvia Sellán, Noam Aigerman, and Alec Jacobson. 2021. Swept volumes via spacetime numerical continuation. *ACM Transactions on Graphics (TOG)* 40, 4 (2021), 1–11.
- Alexei Sourin and A Pasko. 1995. Function representation for sweeping by a moving solid. In *Proceedings of the third ACM symposium on Solid modeling and applications*. 383–391.
- Pengfei Wang, Yuexin Yang, Shuangmin Chen, Shiqing Xin, Changhe Tu, and Wenping Wang. 2025. Swept Volume Computation with Enhanced Geometric Detail Preservation. In *Computer Graphics Forum*, Vol. 44. Wiley Online Library, e70238.
- WP Wang and KK Wang. 1986. Geometric modeling for swept volume of moving solids. *IEEE Computer graphics and Applications* 6, 12 (1986), 8–17.
- Klaus Weinert, Shangjian Du, Patrick Damm, and Marc Stautner. 2004. Swept volume generation for the simulation of machining processes. *International Journal of Machine Tools and Manufacture* 44, 6 (2004), 617–628.
- John D Weld and Ming C Leu. 1990. Geometric representation of swept volumes with application to polyhedral objects. *The International Journal of Robotics Research* 9, 5 (1990), 105–117.
- Zhiqi Xu, Zhiyang Chen, Xiuzi Ye, and Sanyuan Zhang. 2007. Approximate the swept volume of revolutions along curved trajectories. In *Proceedings of the 2007 ACM symposium on Solid and physical modeling*. 309–314.
- Jingzhou Yang and Karim Abdel-Malek. 2005. Approximate swept volumes of NURBS surfaces or solids. *Computer Aided Geometric Design* 22, 1 (2005), 1–26.
- Qingnan Zhou, Eitan Grinspun, Denis Zorin, and Alec Jacobson. 2016. Mesh arrangements for solid geometry. *ACM Transactions on Graphics (TOG)* 35, 4 (2016), 1–15.